Lista de Exercícios

Sistemas de Numeração

98800-04 – Fundamentos de Sistemas Computacionais

Prof. Iaçanã Ianiski Weber

Parte I – Aquecimento: Conceitos Fundamentais

1. Explique, com suas palavras, a diferença entre *número* e *numeral*. Dê um exemplo de um mesmo número representado por três numerais diferentes.

2. Qual a diferença fundamental entre um sistema de numeração **posicional** e um **não-posicional**? Cite um exemplo de cada.

3. No sistema romano (não-posicional), o símbolo `X` sempre vale 10, independentemente de onde aparece. Já no sistema decimal, o dígito `3` pode valer 3, 30, 300, etc. Explique por que essa propriedade torna os sistemas posicionais mais eficientes para cálculos aritméticos.

4. Quantos símbolos diferentes são usados em cada uma das bases a seguir? Liste-os.

a) Base 2 (binário)

b) Base 8 (octal)

c) Base 10 (decimal)

d) Base 16 (hexadecimal)

5. Complete a frase: "Com N bits, é possível representar no máximo _____ coisas distintas." Justifique.

Parte II – Representação Polinomial e Conversões Básicas

6. Escreva a **forma polinomial** de cada número abaixo e calcule seu valor em decimal:

a) $(10110)_2$

b) $(1101\,0011)_2$

c) $(537)_8$

d) $(1A3)_{16}$

e) $(7F)_{16}$

f) $(FF)_{16}$

7. Converta os seguintes números **decimais para binário**, usando divisões sucessivas. Mostre todo o processo.

a) 25_{10}

b) 100_{10}

c) 255_{10}

d) 197_{10}

e) 1024_{10}

f) 2023_{10}

8. Converta os seguintes números **decimais para hexadecimal**. Você pode usar o método que preferir (direto ou passando por binário).

a) 31_{10}
b) 200_{10}
c) 1000_{10}
d) 4095_{10}
e) 65535_{10}
f) 48879_{10}

9. Converta os seguintes números **binários para hexadecimal**, agrupando em *nibbles*:

a) $1011\,0110_2$
b) $1111\,1111_2$
c) $10\,1010\,1010_2$
d) $1\,0000\,0000\,0000_2$

10. Converta os seguintes números **hexadecimais para binário**:

a) `0xCA`
b) `0xFE`
c) `0xDEAD`
d) `0xBEEF`
e) `0xCODE`
f) `0xCAFE`

Dica

Percebeu algo curioso nos números hexadecimais acima? Programadores adoram criar "palavras" usando dígitos hexadecimais (A–F). Esses números são chamados de *hexspeak*. Alguns exemplos famosos: `0xDEADBEEF`, `0xCAFEBABE` (assinatura de arquivos Java `.class`), `0xFEEDFACE` (cabeçalho de binários Mach-O no macOS).

11. Realize a seguinte cadeia de conversões e verifique que o valor final é igual ao inicial:

$$173_{10} \longrightarrow \text{(binário)} \longrightarrow \text{(hexadecimal)} \longrightarrow \text{(decimal)}$$

Parte III – Nibbles, Bytes e Potências de 2

12. Preencha a tabela abaixo (as primeiras linhas estão como exemplo):

Decimal	Hexadecimal	Binário (4 bits)
0	0	0000
1	1	0001
2	2	0010
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		

13. Responda:

a) Quantos valores distintos um *nibble* (4 bits) pode representar?

b) Quantos valores distintos um *byte* (8 bits) pode representar?

c) Quantos dígitos hexadecimais são necessários para representar um *byte*?

d) E para representar uma *word* de 32 bits?

14. Sem fazer cálculos, determine rapidamente o valor decimal de:

a) 2^{10}

b) 2^{16}

c) 2^{20}

d) 2^{32}

Dica

Memorizar as potências de 2 é essencial em computação. Lembre: $2^{10} = 1024 \approx 10^3$ (kilo), $2^{20} \approx 10^6$ (mega), $2^{30} \approx 10^9$ (giga).

15. Um HD é anunciado como tendo 1 TB (terabyte, 10^{12} bytes). Qual o tamanho real em TiB (tebibytes, 2^{40} bytes)? Por que existe essa discrepância entre fabricantes de HD e sistemas operacionais?

Parte IV – Bits Representam Qualquer Coisa

16. Qual é o número mínimo de bits necessários para representar **exclusivamente** os 26 caracteres do alfabeto português (sem acentos)? Quantos caracteres adicionais poderiam ser representados sem bits extras?

17. A tabela ASCII usa 7 bits (arredondados para 1 byte na prática). Quantos caracteres distintos 7 bits podem representar? Por que isso foi suficiente para o inglês, mas não para o português?

18. O padrão Unicode utiliza diferentes codificações: UTF-8, UTF-16 e UTF-32.

a) Quantos caracteres distintos o UTF-32 pode representar *teoricamente*?

b) Pesquise: quantos *code points* estão efetivamente atribuídos na versão mais recente do Unicode? O que explica a diferença?

19. Códigos de cor HTML usam 24 bits (3 bytes), na forma `#RRGGBB`, onde cada componente (R, G, B) usa 1 byte.

a) Quantas cores distintas essa codificação permite?

b) O branco puro é `#FFFFFF`. Qual é esse valor em decimal?

c) O vermelho puro é `#FF0000`. Qual o valor decimal de cada componente (R, G, B)?

d) Invente uma cor, escreva seu código hexadecimal e calcule o valor decimal equivalente.

20. Um endereço IPv4 usa 32 bits. Um endereço IPv6 usa 128 bits.

a) Quantos endereços IPv4 existem? Expresse como potência de 2 e também em notação decimal aproximada.

b) Quantos endereços IPv6 existem? Expresse como potência de 2.

c) A população mundial é de aproximadamente 8×10^9 pessoas. Quantos endereços IPv6 cada pessoa poderia ter, em média? (Apresente o resultado como potência de 2 ou de 10.)

Parte V – Aritmética Binária (Unsigned)

21. Realize as seguintes **adições em binário** (unsigned, 8 bits). Indique se houve overflow.

a) $0101\,0011 + 0010\,1100$

b) $1100\,0011 + 0011\,1101$

c) $1111\,0000 + 0001\,0001$

d) $1011\,1011 + 0111\,0110$

22. Realize as seguintes **subtrações em binário** (unsigned, 8 bits):

a) $1001\,0110 - 0011\,0101$

b) $1111\,1111 - 0000\,0001$

c) $1000\,0000 - 0000\,0001$

d) $0101\,0101 - 0010\,1010$

23. Qual é o maior número inteiro sem sinal (*unsigned*) que pode ser representado com:

a) 4 bits?

b) 8 bits?

c) 16 bits?

d) 32 bits?

e) n bits?

24. Um contador binário de 4 bits começa em 0000 e incrementa de 1 em 1. Escreva a sequência completa de valores até voltar a 0000 (após overflow). Quantos incrementos foram necessários?

Parte VI – Números com Sinal: Sinal-Magnitude

25. Usando **sinal-magnitude com 8 bits**, represente:

a) $+45$

b) -45

c) $+127$

d) -127

e) $+0$

f) -0

26. Na representação sinal-magnitude com n bits:

a) Qual é o intervalo de valores representáveis?

b) Quantas representações distintas existem para o zero?

c) Por que essa duplicidade do zero é problemática para circuitos de comparação?

27. Cite pelo menos duas razões pelas quais sinal-magnitude é raramente usado em computadores de propósito geral.

Parte VII – Complemento de Um

28. Em **complemento de um com 8 bits**, represente:

a) $+33$

b) -33

c) $+100$

d) -100

29. Converta os seguintes valores em **complemento de um (8 bits) para decimal**:

a) 0101 1010

b) 1010 0101

c) 0111 1111

d) 1000 0000

30. Compare sinal-magnitude e complemento de um: ambos têm o problema dos dois zeros. Mostre, para 4 bits, quais são as duas representações do zero em cada sistema.

Parte VIII – Complemento de Dois

31. Usando **complemento de dois com 8 bits**, represente os seguintes valores. Mostre o processo (complementar + somar 1):

a) $+23$

b) -23

c) $+1$

d) -1

e) $+127$

f) -128

32. Converta os seguintes números em **complemento de dois para decimal**:

a) 0110_2 (4 bits)

b) 1101_2 (4 bits)

c) $0110\,1111_2$ (8 bits)

d) $1101\,1011\,0001\,1100_2$ (16 bits)

Dica

Para converter de complemento de dois para decimal, use a fórmula: o bit mais significativo tem peso -2^{N-1} e os demais têm pesos $+2^{N-2}, \ldots, +2^1, +2^0$.

33. Usando **7 bits**, escreva as representações de 23 e -23 em:

a) Sinal-magnitude

b) Complemento de dois

34. Em complemento de dois com 12 bits:

a) Qual é o maior número positivo representável? (em binário e decimal)

b) Qual é o número negativo de maior magnitude? (em binário e decimal)

35. Generalize: em complemento de dois com n bits:

a) Qual é o maior positivo? Expresse como potência de 2.

b) Qual é o menor negativo? Expresse como potência de 2.

c) Quantos números positivos, negativos e zeros existem?

36. Expresse o valor -22 como inteiro em complemento de dois usando:

a) 8 bits

b) 16 bits

c) 32 bits

Você percebe um padrão? Esse fenômeno é chamado **extensão de sinal** (*sign extension*). Descreva-o em uma frase.

37. Explique por que, em complemento de dois com n bits, o valor -2^{n-1} **não** tem um inverso (positivo) representável.

Parte IX – Overflow em Complemento de Dois

38. Descreva as condições que indicam que ocorreu **overflow** ao somar dois números em complemento de dois. (Dica: analise os sinais dos operandos e do resultado.)

39. Os números a seguir estão em complemento de dois com **4 bits**. Realize as operações, converta operandos e resultados para decimal e indique se houve overflow:

a) $0011 + 1100$

b) $0111 + 1111$

c) $1110 + 1000$

d) $0110 + 0010$

e) $0101 + 0100$

f) $1001 + 1010$

g) $0111 + 0001$

h) $1000 + 1111$

40. Considere a soma em 4 bits (signed):

$$1100_2 + 0110_2$$

a) Interprete como **unsigned** e verifique se há overflow.

b) Interprete como **signed (complemento de dois)** e verifique se há overflow.

c) Explique por que a mesma operação pode gerar overflow em uma interpretação e não na outra.

41. Um programador armazena a idade de uma pessoa em uma variável `int8_t` (inteiro com sinal de 8 bits em complemento de dois). Qual é a maior idade que pode ser armazenada? O que aconteceria se ele tentasse armazenar a idade de Jeanne Calment, que viveu 122 anos?

Parte X – Problemas Aplicados e Criativos

42. O Enigma do Odômetro Binário

Imagine um odômetro de carro que, em vez de dígitos decimais, usa 16 bits binários (unsigned).

a) Qual a quilometragem máxima que esse odômetro pode registrar?

b) Se o carro roda 50 km por dia, após quantos dias o odômetro "zera" (overflow)?

c) Se o odômetro usasse 32 bits, quantos anos levariam para zerar, nas mesmas condições?

43. Mensagem Secreta

Uma mensagem foi codificada convertendo cada letra para sua posição no alfabeto (A=1, B=2, ..., Z=26) e depois para binário com 5 bits.

Decodifique: `01000 01001 01110 00001 10010 01001 01111`

44. O Bug do Pac-Man

No jogo original *Pac-Man* (1980), o contador de nível era armazenado em um único byte (8 bits, unsigned). Quando o jogador chegava ao nível 256, ocorria um famoso *bug*.

a) Explique, do ponto de vista de sistemas de numeração, o que aconteceu no nível 256.

b) Qual seria o maior nível possível se o contador usasse 16 bits?

c) E se usasse um `int8_t` (8 bits com sinal em complemento de dois)?

45. Nuclear Gandhi

Na série de jogos *Civilization*, existe uma lenda urbana de que o líder Gandhi, normalmente pacífico, se tornava extremamente agressivo. A "agressividade" de Gandhi começava em 1 (de 0 a 255, armazenada em 1 byte unsigned). Ao adotar a democracia, a agressividade era reduzida em 2.

a) Calcule $1 - 2$ em aritmética de 8 bits unsigned. Qual valor resultaria?

b) Explique o comportamento inesperado usando o conceito de overflow/underflow.

c) Se a variável fosse do tipo `int8_t` (signed), qual seria o resultado de $1 - 2$? O bug aconteceria?

46. O Bug do Ano 2038

Muitos sistemas Unix armazenam timestamps como o número de segundos desde 1º de janeiro de 1970 ("*Unix epoch*"), usando um inteiro com sinal de 32 bits (complemento de dois).

a) Qual é o maior valor positivo que um `int32_t` pode armazenar?

b) Quantos segundos há em um ano (considere 365,25 dias)?

c) Em que ano aproximado o contador irá "estourar" (overflow)?

d) Qual é a solução adotada pelos sistemas modernos?

47. Endereços de Memória

Um processador usa endereços de 32 bits para acessar a memória RAM.

a) Quantos bytes de memória podem ser endereçados?

b) Converta essa quantidade para GiB (2^{30} bytes).

c) Explique por que sistemas operacionais de 32 bits têm um limite prático de 4 GiB de RAM.

d) Processadores de 64 bits resolvem esse problema. Quantos bytes poderiam, em teoria, ser endereçados? É um valor prático?

48. Placas de Veículos

As placas de veículos no Brasil (padrão Mercosul) têm o formato `LLLNLNN`, onde L = letra (26 possibilidades) e N = dígito (10 possibilidades).

a) Quantas placas distintas esse formato permite?

b) Quantos bits seriam necessários para codificar cada placa como um número binário único?

49. Pixel Art

Uma imagem em escala de cinza usa 8 bits por pixel (0 = preto, 255 = branco).

a) Quantos tons de cinza distintos essa codificação permite?

b) Considere uma imagem de 1920×1080 pixels. Quantos bytes são necessários para armazená-la (sem compressão)?

c) Se cada pixel usasse 24 bits (cor RGB), quantos bytes seriam necessários?

d) Converta as respostas anteriores para MiB (2^{20} bytes).

50. Criptografia e Bases

A codificação *Base64* é amplamente usada na internet para transmitir dados binários como texto. Ela usa 64 símbolos para representar dados.

a) Quantos bits cada "dígito" de Base64 representa?

b) Se você tem 3 bytes (24 bits) de dados, quantos caracteres Base64 são necessários?

c) Por que Base64 é menos eficiente que binário puro? Calcule o overhead percentual.

Parte X-B – Sistema Octal

51. Converta os seguintes números **decimais para octal**, usando divisões sucessivas:

a) 100_{10}

b) 255_{10}

c) 511_{10}

d) 1000_{10}

e) 4096_{10}

f) 8191_{10}

52. Converta os seguintes números **octais para decimal**:

a) $(77)_8$

b) $(377)_8$

c) $(1750)_8$

d) $(644)_8$

e) $(755)_8$

f) $(2000)_8$

53. A conversão entre **binário e octal** é direta, agrupando os bits de 3 em 3 (da direita para a esquerda). Converta:

a) $1101\,0110_2 \longrightarrow$ octal

b) $1\,0000\,0000_2 \longrightarrow$ octal

c) $(375)_8 \longrightarrow$ binário

d) $(17)_8 \longrightarrow$ binário

Dica

Em sistemas Unix/Linux, as permissões de arquivo são expressas em octal. Por exemplo, `chmod 755` significa: dono = 7 (rwx), grupo = 5 (r-x), outros = 5 (r-x). Cada dígito octal corresponde a 3 bits: leitura (r=4), escrita (w=2), execução (x=1).

54. Permissões Unix

No Linux, cada arquivo tem permissões para três categorias: dono, grupo e outros. Cada categoria tem 3 bits: leitura (r=4), escrita (w=2) e execução (x=1). A soma desses valores forma um dígito octal.

a) Qual é a representação binária (9 bits) e octal das permissões `rwxr-xr-`?

b) Um arquivo tem permissão octal `640`. Quais são as permissões de cada categoria?

c) Por que o sistema octal é particularmente conveniente para permissões Unix? (Pense na relação entre 3 bits e 1 dígito octal.)

Parte X-C – Aritmética em Outras Bases

55. Realize as seguintes **adições em hexadecimal** (mostre os *carries*):

a) `0x3A` + `0x1F`

b) `0xFF` + `0x01`

c) `0xAB` + `0xCD`

d) `0x1234` + `0xABCD`

56. Realize as seguintes **subtrações em hexadecimal**:

a) `0xFF` − `0x0A`

b) `0x100` − `0x01`

c) `0xC0` − `0x3F`

d) `0xDEAD` − `0xBEEF`

57. Realize as seguintes **adições em octal**:

a) $(35)_8 + (47)_8$

b) $(177)_8 + (1)_8$

c) $(777)_8 + (1)_8$

d) $(644)_8 + (133)_8$

58. Multiplicação e divisão por potências de 2

Em binário, multiplicar por 2 equivale a deslocar todos os bits uma posição à esquerda (*shift left*), e dividir por 2 equivale a deslocar uma posição à direita (*shift right*).

a) Dado $0010\,1100_2$ (44 em decimal), calcule o resultado de deslocá-lo 1 bit à esquerda. Verifique convertendo para decimal.

b) Dado $0110\,0000_2$ (96 em decimal), calcule o resultado de deslocá-lo 2 bits à direita. Verifique.

c) Generalize: deslocar k bits à esquerda equivale a multiplicar por quanto? E deslocar k bits à direita?

d) Um programador precisa multiplicar um número por 10. Usando apenas shifts e adições, como ele pode fazer isso? (Dica: $10 = 8 + 2$.)

Parte X-D – Números Fracionários em Binário

Assim como no sistema decimal temos casas após a vírgula ($10^{-1}, 10^{-2}, \ldots$), em binário as posições após o ponto binário valem $2^{-1} = 0{,}5$, $2^{-2} = 0{,}25$, $2^{-3} = 0{,}125$, etc.

59. Converta os seguintes números binários fracionários para decimal:

a) $0{,}1_2$

b) $0{,}11_2$

c) $0{,}101_2$

d) $1{,}1_2$

e) $10{,}01_2$

f) $11{,}111_2$

60. Converta os seguintes números decimais fracionários para binário (use até 6 casas binárias):

a) $0{,}5_{10}$
b) $0{,}75_{10}$
c) $0{,}625_{10}$
d) $0{,}1_{10}$
e) $0{,}3_{10}$
f) $0{,}7_{10}$

Dica

Método para converter a parte fracionária: multiplique por 2 repetidamente. A parte inteira de cada resultado é o próximo bit. Exemplo: $0{,}75 \times 2 = 1{,}5$ (bit $= 1$), $0{,}5 \times 2 = 1{,}0$ (bit $= 1$). Logo $0{,}75_{10} = 0{,}11_2$.

61. Você notou que $0{,}1_{10}$ não tem uma representação binária exata (finita)?

a) Explique por que isso acontece.

b) Execute o seguinte teste mental (ou em um computador): em Python, 0.1+0.2 == 0.3? O resultado te surpreende?

c) Que implicações práticas isso tem para softwares financeiros?

Parte X-E – Operações Lógicas Bit a Bit

Além de operações aritméticas, computadores realizam operações **lógicas** bit a bit: AND (&), OR (|), XOR ($\oplus$) e NOT ($\sim$). Cada operação é aplicada individualmente a cada par de bits correspondentes.

62. Calcule os resultados das seguintes operações bit a bit (8 bits):

a) 1010 0110 AND 1100 0011

b) 1010 0110 OR 1100 0011

c) 1010 0110 XOR 1100 0011

d) NOT 1010 0110

63. Máscaras de bits (*bitmasks*)

Máscaras de bits são usadas para extrair, definir ou limpar bits específicos de um número.

a) Dado o byte `0xB7`, aplique a máscara `0x0F` com AND. O que essa operação extrai?

b) Dado o byte `0xB7`, aplique a máscara `0xF0` com AND e desloque 4 bits à direita. O que você obtém?

c) Explique como usar OR para "ligar" o bit 3 (contando de 0, da direita) de um byte, sem alterar os demais bits.

d) Explique como usar AND para "desligar" o bit 5 de um byte, sem alterar os demais.

e) Uma propriedade do XOR: $A \oplus B \oplus B = A$. Explique por que essa propriedade é útil em criptografia simples e em algoritmos de troca de variáveis sem usar uma variável temporária.

64. Swap sem variável temporária

Dados $A = 0101\,1010$ e $B = 1100\,0011$, execute a sequência:

a) $A \leftarrow A \oplus B$

b) $B \leftarrow A \oplus B$

c) $A \leftarrow A \oplus B$

Mostre o valor de A e B após cada passo. O que aconteceu?

Parte X-F – Problemas Extras do Mundo Real

65. O Gangnam Style e o YouTube

Em 2014, o clipe de "Gangnam Style" do PSY ultrapassou 2.147.483.647 visualizações no YouTube. O contador do YouTube parou de funcionar.

a) Reconhece esse número? Qual é sua relação com potências de 2?

b) O YouTube usava `int32_t` para contar visualizações. Qual foi o problema?

c) Após o incidente, o YouTube migrou para `int64_t`. Qual é o novo limite? É suficiente?

66. Endereços MAC

Cada placa de rede possui um endereço MAC único de 48 bits, geralmente escrito como 6 pares hexadecimais: `AA:BB:CC:DD:EE:FF`.

a) Quantos endereços MAC distintos existem? Expresse como potência de 2 e em notação decimal aproximada.

b) Converta o endereço MAC `00:1A:2B:3C:4D:5E` para binário.

c) Quantos dígitos hexadecimais são necessários para representar um endereço MAC? Por quê?

67. Códigos de Barras e QR Codes

Um código de barras EAN-13 (usado em produtos) codifica 13 dígitos decimais.

a) Quantas combinações distintas de 13 dígitos decimais existem?

b) Quantos bits seriam necessários para representar esse mesmo número de combinações em binário puro?

c) Compare com a codificação BCD: quantos bits seriam necessários em BCD para 13 dígitos?

68. GPS e Precisão

Coordenadas GPS são freqüentemente armazenadas como números inteiros com sinal de 32 bits, representando micrograus (milionésimos de grau). Por exemplo, a latitude $-29{,}951\,066°$ seria armazenada como $-29\,951\,066$.

a) A latitude varia de $-90\,000\,000$ a $+90\,000\,000$ micrograus. Esse intervalo cabe em um `int32_t`? Justifique.

b) E a longitude ($-180\,000\,000$ a $+180\,000\,000$ micrograus)? Cabe?

c) Qual é a precisão em metros de 1 micrograu? (Circunferência da Terra $\approx 40\,000$ km.)

69. Cifra XOR

Uma das cifras mais simples em criptografia é o XOR com uma chave.

Uma mensagem de 1 byte `M = 0x48` é cifrada com a chave `K = 0x5A` usando XOR.

a) Calcule o texto cifrado $C = M \oplus K$. Mostre o cálculo em binário.

b) Mostre que aplicar XOR novamente com a mesma chave recupera a mensagem: $M = C \oplus K$.

c) O byte `0x48` corresponde ao caractere 'H' na tabela ASCII. Qual caractere é o texto cifrado?

d) Por que a cifra XOR sozinha não é segura para criptografia real?

70. Capacidade de Armazenamento ao Longo da História

O primeiro disco rígido da IBM (1956, modelo 305 RAMAC) armazenava 5 MB. Hoje, discos de 20 TB são comuns.

a) Converta 5 MB e 20 TB para bits.

b) Quantos bits são necessários para endereçar cada byte do disco de 5 MB?

c) E para o disco de 20 TB?

d) Quantas vezes a capacidade de armazenamento aumentou de 1956 até hoje? Expresse como potência de 2 aproximada.

Parte XI – Desafios (Opcional)

Desafio

71. Demonstre algebricamente que, em complemento de dois com n bits, o inverso aditivo de um número x pode ser obtido complementando todos os bits e somando 1. Ou seja, prove que:

$$-x = \overline{x} + 1$$

onde $\overline{x}$ denota o complemento bit a bit de x.
Dica: comece observando que $x + \overline{x} = 2^n - 1$ (todos os bits em 1).

72. Sistema de base -2 ("negabinário")

Sim, é possível usar bases negativas! Na base -2, os pesos das posições são $(-2)^0 = 1$, $(-2)^1 = -2$, $(-2)^2 = 4$, $(-2)^3 = -8$, etc.

a) Calcule o valor decimal de 11010_{-2}.

b) Represente -6 na base -2.

c) Qual é a vantagem teórica desse sistema em relação ao complemento de dois?

73. BCD (*Binary-Coded Decimal*)

Na codificação BCD, cada dígito decimal é representado individualmente por 4 bits.

a) Represente 9742_{10} em BCD.

b) O BCD é mais ou menos eficiente que binário puro em termos de espaço? Justifique comparando quantos bits são necessários para representar o número 255 em ambos os formatos.

c) Pesquise: em que tipo de aplicação o BCD ainda é utilizado e por quê?

74. O Curioso Caso do -0 no Espaço

Em 1996, o foguete Ariane 5 da Agência Espacial Europeia explodiu 37 segundos após o lançamento. A causa raiz foi a conversão de um número de ponto flutuante de 64 bits para um inteiro com sinal de 16 bits, gerando overflow.

a) Qual é o intervalo de valores de um inteiro com sinal de 16 bits (complemento de dois)?

b) Se a velocidade horizontal do foguete era armazenada como um valor que excedia 32 767, o que aconteceu na conversão?

c) Que lição sobre sistemas de numeração e overflow esse acidente ensina?

75. Soma de Verificação (*Checksum*)

Uma técnica simples de verificação de integridade de dados consiste em somar todos os bytes de uma mensagem (em complemento de um) e usar o resultado como *checksum*. Considere a mensagem formada pelos bytes: `0x4F`, `0x6C`, `0x61`.

a) Some os três bytes em binário (8 bits). Se houver *carry*, some-o de volta ao resultado.

b) Complemente (inverta) todos os bits. Este é o checksum.

c) Ao receber a mensagem junto com o checksum, como o receptor pode verificar a integridade?

76. O Sistema Ternário Balanceado

O computador soviético Setun (1958) usava o sistema **ternário balanceado**, com dígitos $\{-1, 0, +1\}$ (representados como T, 0, 1). As posições valem potências de 3.

a) Calcule o valor decimal de $1T01$ (onde $T = -1$).
Dica: $1 \cdot 3^3 + (-1) \cdot 3^2 + 0 \cdot 3^1 + 1 \cdot 3^0$

b) Represente -13 em ternário balanceado.

c) Qual vantagem esse sistema tem sobre complemento de dois para representar números negativos?

77. Números de Ponto Flutuante – Introdução

O padrão IEEE 754 (precisão simples, 32 bits) divide os bits em três campos:

- 1 bit de sinal (s)
- 8 bits de expoente (e), com *bias* de 127
- 23 bits de mantissa (m), com um "1" implícito

O valor é: $(-1)^s \times 1{,}m \times 2^{(e-127)}$

a) O número `0 10000010 10100000000000000000000` está em IEEE 754. Qual é seu valor decimal?

b) Represente $-6{,}5$ em IEEE 754 (32 bits).

c) Por que números de ponto flutuante podem representar valores muito maiores que inteiros de 32 bits, mas com menos precisão?

78. Código de Gray

O Código de Gray é um sistema binário onde dois valores consecutivos diferem em apenas 1 bit. Ele é usado em encoders rotatórios e na correção de erros.
A conversão de binário para Gray é: $G_i = B_i \oplus B_{i+1}$ (XOR entre bits adjacentes, onde $B_n = 0$ para o bit mais significativo).

a) Construa a tabela do Código de Gray para 4 bits (0 a 15).

b) Verifique que valores consecutivos diferem em exatamente 1 bit.

c) Converta o binário 1011 para Código de Gray.

d) Por que o Código de Gray é preferível ao binário natural em encoders rotatórios? (Pense no que acontece quando múltiplos bits mudam simultaneamente.)

79. Paradoxo do Aniversário em Hashes

Funções hash mapeiam dados de tamanho arbitrário para um valor de tamanho fixo. O "paradoxo do aniversário" implica que colisões (dois inputs com o mesmo hash) são prováveis após aproximadamente $\sqrt{2^n}$ tentativas, onde n é o número de bits do hash.

a) Para um hash de 32 bits, após quantas tentativas uma colisão se torna provável? Expresse como potência de 2.

b) Para um hash de 256 bits (SHA-256), calcule o mesmo valor.

c) A rede Bitcoin processa $\sim 10^{20}$ hashes por ano. Quantos anos seriam necessários para encontrar uma colisão em SHA-256?

"Existem 10 tipos de pessoas no mundo: as que entendem binário e as que não entendem."